

Thinking with Machines: Agents and RL for Financial Applications

Sanjiv Das, <https://srdas.github.io>

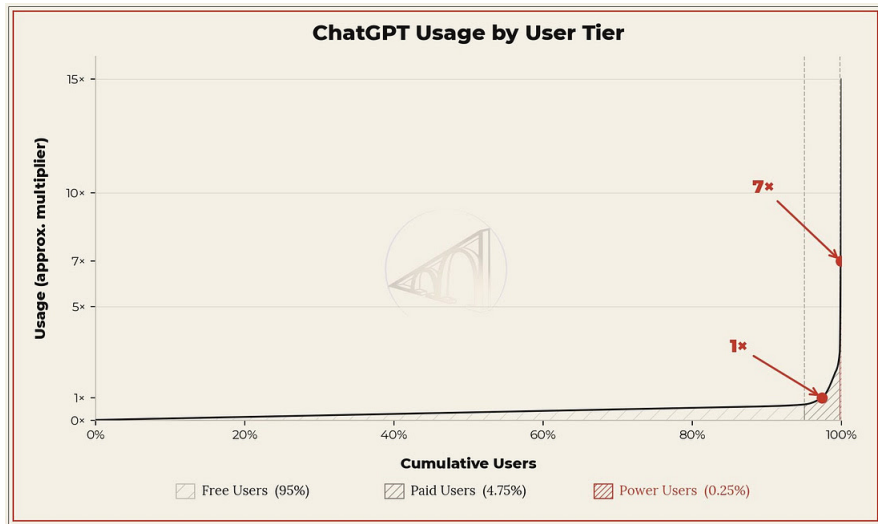
(Joint work with Harshad Khadilkar, Sukrit Mittal, Daniel Ostrov, Jacob Sisk)

Santa Clara University

MIT Sloan

- ① From Chat to Agentic AI
- ② Reinforcement learning for wealth management problems
- ③ Using agents to democratize RL solutions for wealth management
- ④ Guidance for Agentic AI in Finance

Capability Overhang



Capability Overhang

- ① There are many chat apps:
 - ChatGPT: <https://chatgpt.com>
 - Gemini: <https://gemini.google.com/app>
 - Perplexity: <https://www.perplexity.ai>
 - t3: <https://t3.chat>
 - Mercury2, from Inception Labs: <https://chat.inceptionlabs.ai/>
- ② Most people use ChatGPT, etc., as better search. They're missing 99% of the value.
- ③ The AI companies call this a capability overhang. The models have been able to do far more than anyone uses them for, but nobody noticed because the tooling hadn't caught up. But now we are democratizing Agentic AI.
<https://cdn.openai.com/pdf/openai-ending-the-capability-overhang.pdf>

Three Components of Agentic AI Infrastructure

- **Models** are the Core "Brains": These underlying systems, such as Claude-Opus-4.8, Gemini 3 Pro, or GPT-5.5, determine the platform's actual intelligence and reasoning capabilities. They are the engine behind the scenes that dictates how effectively the AI can write, code, or analyze complex data sets.
- **Apps** are the User Interface: Apps are the specific products and websites, like chatgpt.com or Claude Cowork, that allow you to interact with the underlying models. They act as the bridge between the user and the AI, providing a functional space to perform tasks on a phone, desktop, or browser.
- **Harnesses** Enable Action: A harness is the system that allows an AI model to use external **tools**, browse the web, and execute multi-step workflows. While apps often include a built-in harness, standalone versions like OpenClaw allow models to take direct action on a computer to complete complex projects from scratch.

Agent: A model with a harness and tools.

What changed for the bots? (and for humans ...)

- 1 **The models keep getting better** — and are improving themselves. — OpenAI used early Codex to debug its own training runs. The model helped build the model. Better models → better tools → better models. The flywheel is rapid.
- 2 **Models escaped the chat box.** The biggest rate limiter to AI value hasn't been intelligence — it's been the interface. That's over. Claude is now working directly in Excel, Notion, Linear, Slack — not as a sidebar, but as the analyst.
- 3 **Task length has exploded.** Research from METR shows autonomous task horizons doubling every four months. At 30 minutes: code snippets. At 5 hours: module refactoring. At multi-day: full codebase audits. Each doubling unlocks exponentially more of the total automation pie.
- 4 **Agents work in teams.** Cursor built an agent harness that orchestrates thousands of agents simultaneously. Their FastRender experiment had 30,000 commits, 2,000 concurrent agents, and a million+ lines of Rust. With a planner/worker architecture with specialized roles.

Quick Agents

The screenshot displays the JupyterLab web interface in a browser window. The left sidebar contains a 'QuickAgent list' with three agents: 'QuickAgent', 'Joker', and 'MathQ'. Each agent has a 'Purpose', 'Tools', 'Search', and 'Skills' section. The 'Joker' agent's purpose is to write a standup comedy joke. The 'MathQ' agent's purpose is to generate and solve math questions. The 'Researcher' agent's purpose is to read files, formulate research questions, and produce a LaTeX paper. The main area shows the 'JupyterLab settings' panel, which includes sections for 'Chat model' (configured with 'bedrock/us.anthropic.claude-opus-4-7'), 'Model parameters', 'Secrets and API keys' (showing 'No secrets configured'), and a 'Launcher' tab.

QuickAgent list

- QuickAgent**
Saved Quick Agents:
- Joker**
 - Purpose:** Agent that writes a standup comedy joke in a few sentences from the provided file or URL
 - Tools:** execute, read_file, write_file, edit_file, ls, glob, grep, python_repl, web_fetch, todo
 - Search:** duckduckgo_search
 - Skills dir:** /Users/sanjvda/CODE/skill-collection/comedy_writer
- MathQ**
 - Purpose:** Generate and solve math questions that test deep conceptual understanding rather than mechanical computation or memorized formulas. These questions should reward creativity, multi-perspective thinking, and the ability to connect seemingly unrelated ideas.
 - Tools:** execute, read_file, write_file, edit_file, ls, glob, grep, python_repl, web_fetch, todo
 - Search:** none
 - Skills dir:** /Users/sanjvda/CODE/skill-collection/math_questions
- Researcher**
 - Purpose:** Agent to read all files in a folder, formulate good research questions, perform empirical analysis, present results with visualizations, generate a reproducible Jupyter notebook, and produce a LaTeX paper
 - Tools:** execute, read_file, write_file, edit_file, ls, glob, grep, python_repl, web_fetch, todo
 - Search:** none
 - Skills dir:** /Users/sanjvda/CODE/skill-collection/researcher

Type a chat message, @ to mention...

JupyterLab settings

- Chat model**
Configure the language model used by Jupyterlab in chats.
You may provide any model ID accepted by LiteLLM. Details can be found from the [LiteLLM documentation](#).
bedrock/us.anthropic.claude-opus-4-7
[Update Chat Model](#)
- Model parameters**
Configure additional parameters for the language model.
[Add A Custom Model Parameter](#)
- Secrets and API keys**
This section shows the secrets set in the .env file at the workspace root. For most chat models, an API key secret in .env is required for Jupyterlab to reply in the chat. See the [documentation](#) for information on which API key is required for your model provider.
Click "Add secret" to add a secret to the .env file. Secrets can also be updated by editing the .env file directly in JupyterLab.
No secrets configured
Click "Add secret" to add an API key and start using Jupyterlab with your preferred model provider.
[+ Add secret](#)

<https://github.com/srdas/jupyter-ai-quickagent>

Multi-Agent Systems

- CrewAI: <https://www.crewai.com/>, docs: <https://docs.crewai.com/en/introduction>
- Agno: <https://github.com/agno-agi/agno>, docs: <https://docs.agno.com/introduction>
- Langchain DeepAgents: <https://github.com/langchain-ai/deepagents>, docs: <https://docs.langchain.com/oss/python/deepagents/overview>
- Examples: https://srdas.github.io/NLPBook/Multi_Agents.html

Agno Agent

```
1 from agno.agent import Agent
2 from agno.models.openai import OpenAIChat
3 from agno.tools.duckduckgo import DuckDuckGoTools
4 from agno.tools.yfinance import YFinanceTools
5 from agno.team import Team
6
7 web_agent = Agent(
8     name="Web Agent",
9     role="Search the web for information",
10    model=OpenAIChat(id="gpt-4.1"),
11    tools=[DuckDuckGoTools()],
12    instructions="Always include sources",
13    # show_tool_calls=True,
14    markdown=True,
15 )
16
17 finance_agent = Agent(
18     name="Finance Agent",
19     role="Get financial data",
20     model=OpenAIChat(id="gpt-4.1"),
21     # tools=[YFinanceTools(stock_price=True, analyst_recommendations=True, company_info=True)],
22     tools=[YFinanceTools()],
23     instructions="Use tables to display data",
24     # show_tool_calls=True,
25     markdown=True,
26 )
27
28 agent_team = Team(
29     # mode="coordinate",
30     members=[web_agent, finance_agent],
31     model=OpenAIChat(id="gpt-4.1"),
32     # success_criteria="A comprehensive financial news report with clear sections and data-driven insights.",
33     instructions=["Always include sources", "Use tables to display data"],
34     # show_tool_calls=True,
35     markdown=True,
36 )
37
38 agent_team.print_response("What's the market outlook and financial performance of AI semiconductor companies?", stream=True)
```

Earnings Call Analysis

Message

Summarize the earnings call for Amazon Q1 2026

Team Tool Calls

- `delegate_task_to_member(member_id=web-agent, task=Find a reliable and up-to-date summary of Amazon's Q1 2026 earnings call. Provide key financial figures (revenue, net income, EPS), highlights from management commentary, notable business developments, and overall market reaction if available. Include credible sources.)`

Response (13.5s)

Here is a summary of Amazon's Q1 2026 earnings call:

Metric	Q1 2026 Result	Year-Over-Year Change
Revenue	\$181.5 billion	+17% (15% excluding FX impact)
Net Income	Not explicitly stated	[Further research needed]
Earnings Per Share (EPS)	Not explicitly stated	[Further research needed]
AWS Revenue	Significant growth	+28% (beat analyst estimates)

Management Highlights & Business Developments

- **Amazon Web Services (AWS):** Strong performance, up 28% year-over-year, exceeding analysts' expectations. Cloud and infrastructure spending continues to drive segment growth.
- **LEO Satellite Project:** Remains on track for Q3 2026 launch. Management anticipates a \$1 billion year-over-year cost increase in Q2 for North America due to increased satellite manufacturing and deployment.
- **Guidance:** Management referenced ongoing risks from foreign exchange volatility, energy prices, and global economic/geopolitical uncertainty, but maintained a positive outlook for profitability.

Market Reaction

- Investors responded positively to AWS results, with after-hours trading reflecting a modest share price increase.

Sources

- [Amazon Q1 2026 Earnings Release](#)
- [CNBC: Amazon Q1 2026 Earnings](#)

Market Outlook

Message

What's the market outlook for the S&P 500 as of the end of 2026

Team Tool Calls

- `delegate_task_to_member(member_id=web-agent, task=Research consensus forecasts, major bank or analyst projections, and reputable financial media commentary on the S&P 500 market outlook for the end of 2026. Summarize key forecasts, reasoning, and risk factors. Include relevant sources and, if available, provide any numeric targets or ranges.)`

Response (21.6s)

Here's a summary of the S&P 500 market outlook as of the end of 2026, based on major banks, analysts, and reputable financial sources:

Source	Year-End 2026 Target	Key Reasoning	Major Risks	Source Link
Goldman Sachs	8,000	AI-led earnings growth; strong capex.	Fed policy, earnings disappointment.	Goldman Sachs
Wall Street Consensus	7,100–8,000	Broad earnings growth; sector rotation.	Geopolitics, sector concentration.	GAR Capital, Investing.com
Conservative Forecasters	6,200–6,800	Moderate earnings, cautious valuations.	Global risks, Fed uncertainty.	StockNewsPlus
Analyst EPS Consensus	\$306–\$340	Earnings driven by AI adoption.	Tech disappointment, macro shocks.	Goldman Sachs, Investing.com

Key Themes:

- Robust earnings growth fueled by artificial intelligence and technology investment.
- Potential support from lower Fed rates if inflation remains controlled.
- Possible risks from earnings disappointment, geopolitics, Fed policy shifts, and market sector concentration.

Risks Highlighted:

- Political/government instability (U.S. elections, global tensions)
- AI-driven earnings failing to materialize
- Valuation concerns with persistent inflation or higher rates

Agent Swarms for Portfolio Management

- Ang, Azimbayev, Kim (2026): “The Self Driving Portfolio: Agentic Architecture for Institutional Asset Management”,
<https://ideas.repec.org//p/arx/papers/2604.02279.html>

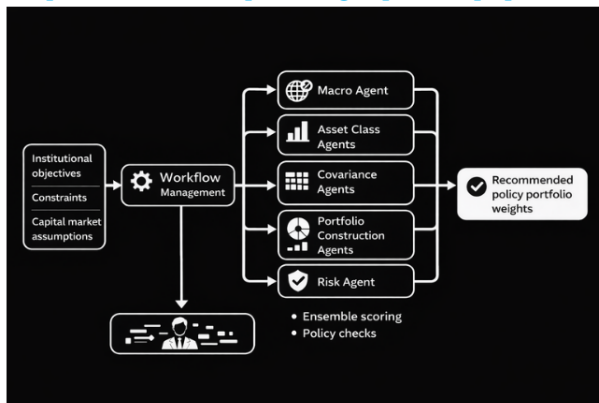


Exhibit 1: Agentic Strategic Asset Allocation Architecture

- A new age of non-deterministic programming (NDP)

RL for Goals-Based Wealth Management

Goals-Based Wealth Management (GBWM)

Investors have multiple goals they want to attain like...

- Purchasing a **Car**
- Going on a **Trip**
- Having enough money for **Retirement**
- Giving money to **Charity**
- Etc.

Each goal has three attributes:

- 1 The **time** at which the goal is available. (E.g., the investor wants a car **five** years from now.)
- 2 The **cost** of attaining the goal. (E.g., the car will cost **\$40,000**.)
- 3 The **importance** of the goal to the investor (measured by the **utility** of the goal to the investor).

Case Studies: Samples

- ① Single-goal: Planning to purchase a house in 10 years.
 - ② Multi-goal: Planning to purchase a car in 3 years, Europe vacation in 5 years, and house in 15 years.
 - ③ Retirement-goal: Planning the annual income for the retirement tenure, till life expectancy.
- Testing horizons go up to 100 years to evaluate out of distribution capability.
 - Number of goals varies from 1 to 60 with a mean of 38.

Table 9: Description of test cases with infusions. The base amount of infusion for odd-numbered test cases is $I = \frac{W(0)}{W(T-1)}$, rounded up to the nearest integer.

Case	T	$W(0)$	Non-zero goals $[t, C(t), U(t)]$	Non-zero infusions $[t, I(t)]$
34	10	100	[10, 150, 1]	[1, 10]
35	10	100	[10, 200, 1]	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
36	10	100	[10, 400, 1]	[1, 10]
37	40	100	[40, 600, 1]	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
38	40	100	[40, 1200, 1]	[6, 12]
39	40	100	[40, 2400, 1]	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
40	100	100	[100, 50000, 1]	[21, 19]
41	100	100	[100, 500000, 1]	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
42	100	100	[100, 200000, 1]	[27, 22]
43	3	100	[2, 75, 0.9], [3, 75, 1]	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
44	20	100	[10, 200, 1.3], [20, 500, 1]	[6, 12]
45	20	100	[15, 200, 1.3], [20, 300, 1]	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
46	35	100	[5, 50, 1], [25, 500, .5], [35, 1000, 1]	[13, 15]
47	60	100	[15, 300, .7], [60, 6000, 1.2], [45, 5000, .2], [60, 20000, 1]	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
48	25	100	[3, 30, .2], [5, 70, .3], [8, 70, .3], [25, 1000, 1]	[11, 14]
49	40	100	[10, 150, 1.5], [40, 600, 1], [35, 500, 1], [40, 600, 1]	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
50	20	100	$[t, 15, 1]$ for $t \in \{2, 4, 6, \dots, 20\}$	[10, 13]
51	20	100	$[t, 25, 1]$ for $t \in \{2, 4, 6, \dots, 20\}$	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
52	20	100	$[t, 50, 1]$ for $t \in \{2, 4, 6, \dots, 20\}$	[11, 14]
53	20	100	$[t, 75, 1]$ for $t \in \{2, 4, 6, \dots, 20\}$	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
54	60	100	$[t, 20, 1]$ for $t \in \{1, 2, 3, \dots, 60\}$	[58, 31]
55	60	100	$[t, t, 1]$ for $t \in \{1, 2, 3, \dots, 60\}$	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
56	60	100	$[t, t, 100 + t]$ for $t \in \{1, 2, 3, \dots, 60\}$	[41, 34]
57	60	100	$[t, t, 100 - t]$ for $t \in \{1, 2, 3, \dots, 60\}$	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
58	60	100	$[t, 60 - \frac{1}{2}]$ for $t \in \{1, 2, 3, \dots, 60\}$	[45, 38]
59	60	100	$[t, 60 - \frac{1}{2}, 100 + t]$ for $t \in \{1, 2, \dots, 60\}$	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
60	60	100	$[t, 60 - \frac{1}{2}, 100 - t]$ for $t \in \{1, 2, \dots, 60\}$	[49, 43]
61	30	100	[3, 35, 1.5], [6, 35, 1.3], [9, 5, 0.4], [12, 50, 1], [15, 15, 0.7], [18, 3, 0.3], [21, 45, 0.6], [24, 120, 0.9], [27, 170, 1.1], [30, 160, 1]	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
62	16	12	[16, 34.25, 26]	[14, 3]
63	16	21.63	[8, 18.50, 18], [16, 34.25, 26]	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
64	16	38.99	[4, 13.60, 14], [8, 15.87, 16], [12, 25.18, 22], [16, 34.25, 26]	[15, 6]
65	16	70.27	[2, 11.66, 12], [4, 13.60, 14], [6, 15.87, 16], [8, 18.50, 18], [10, 21.59, 20], [12, 25.18, 22], [14, 29.37, 24], [16, 34.25, 26]	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
66	16	126.67	[1, 10.8, 11], [2, 11.66, 12], [3, 12.60, 13], [4, 13.60, 14], [5, 14.69, 15], [6, 15.87, 16], [7, 17.14, 17], [8, 18.50, 18], [9, 19.99, 19], [10, 21.59, 20], [11, 23.32, 21], [12, 25.18, 22], [13, 27.20, 23], [14, 29.37, 24], [15, 31.72, 25], [16, 34.25, 26]	[16, 21]

- GBWM changes the notion of risk from **portfolio volatility** to the risk of **not meeting one's goals**. E.g.: the risk of running out of money in retirement.
- These two versions of risk are counterfactual to each other. Taking too little or too much risk can make reaching goals less probable.
- Risk defined as the probability of not reaching one's goals is less volatile than portfolio value, bringing some reassurance to investors.

Basic Mathematics of GBWM

Example for Goals:

- 1 Investment Tenure: 10 years.
- 2 Initial Wealth: \$400,000
- 3 Target Wealth: \$500,000
- 4 Target Probability: 80%

Wealth Dynamics:

$$W(t) = W(0) \exp \left[\left(\mu - \frac{\sigma^2}{2} \right) t + \sigma \sqrt{t} \cdot Z \right]$$

$$Z \sim N(0, 1)$$

$$\mu = \frac{1}{2} \sigma^2 + \frac{z_0}{\sqrt{t}} \sigma + \frac{1}{t} \ln \left(\frac{W(t)}{W(0)} \right)$$

where z_0 is defined such that the target probability equals the normal CDF $\Phi(z_0)$

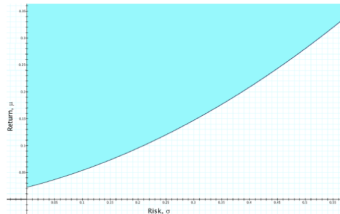
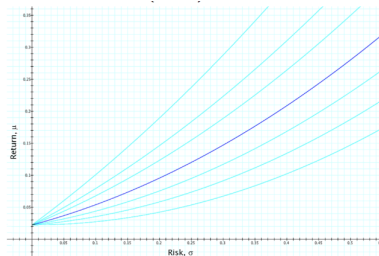
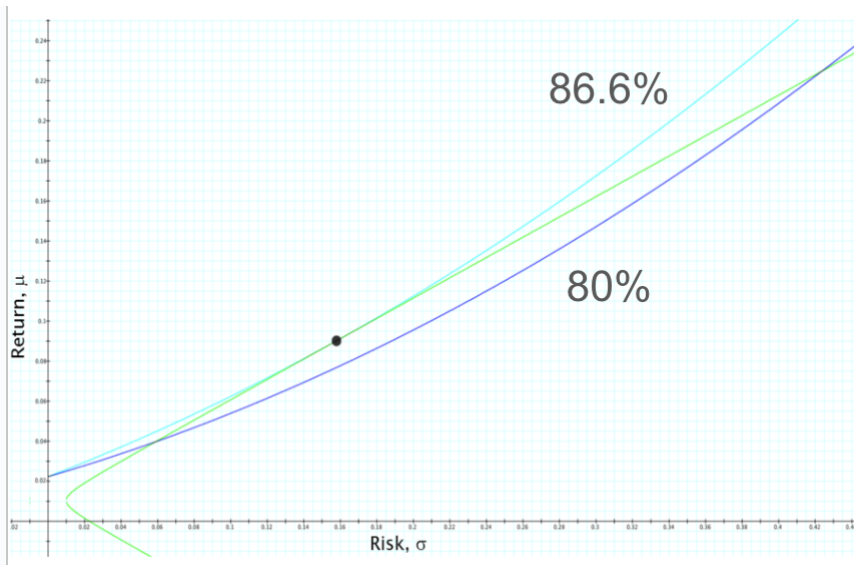


Figure 3: A Goal Region Example. The goal region (in light blue) in this figure corresponds to a base case where an investor has an Investment Tenure of 10 years, an Initial Wealth of \$400,000, a Target Wealth of \$500,000, and a Target Probability of 80%. The boundary of the goal region is the 80% Goal Probability Level Curve (GPLC), which is defined as the curve for which any (σ, μ) pair above the curve corresponds to a portfolio with an 80% or higher chance of the investor attaining their Target Wealth at the end of their investment tenure.

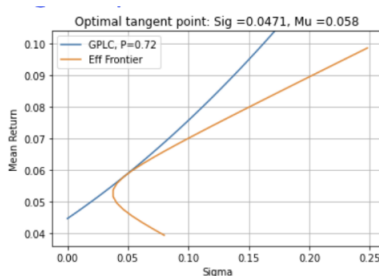


The Optimal Goal Probability Point

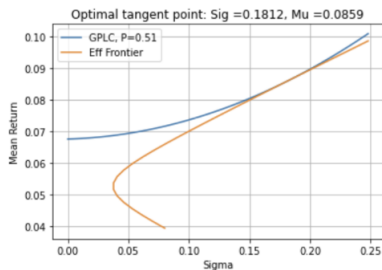


Solution Examples

Initial wealth is $W(0) = 100$, target wealth is $W(t) = 125$, $t = 5$ years, and the optimal goal probability is 72%.



Initial wealth is $W(0) = 100$, target wealth is $W(t) = 150$, $t = 6$ years, and the optimal goal probability is 51%.



Optimizing GBWM Solutions

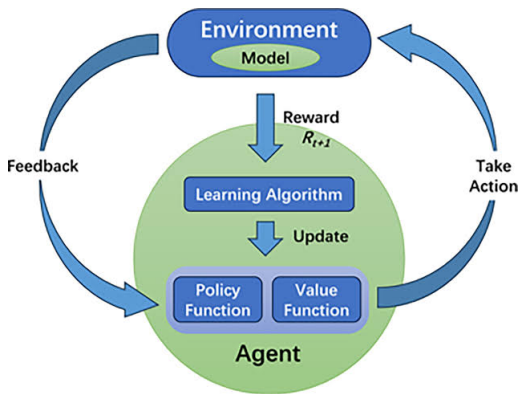
Investors want to optimize...

- ...the number of their attained goals, weighted by their importance.
- That is, we look to maximize **the sum of the attained goals' utilities**.

To do this, we must determine the investor's optimal dynamic strategy, meaning...

- ① determining the optimal **investment portfolio** to be in now, knowing it can change throughout the future.
- ② determining the optimal **goal-taking choice** for a currently available goal, meaning whether it is best to attain the goal (and its utility) or forgo the goal (to save money, increasing the probabilities for attaining later goals).

Quick Overview of Reinforcement Learning (RL)



- Similar to Atari games.
- For deep RL, both the Policy and Value functions are neural nets.
- Policy functions takes actions, Value function evaluates the actions.
- The Learning Algorithm trains the Policy and Value functions using rollouts in batches using Rewards as drivers.

PPO Loop



PPO is an **Actor-Critic** algorithm. PPO solves a major problem in RL: **stability**.

It uses two neural networks:

- 1 **Actor**: Decides which action to take (the policy)
- 2 **Critic**: Predicts the rewards the agent will get (the value function).

Steps:

- 1 **Collection**: Run the current policy for T steps to collect transitions (state, action, reward, probability).
- 2 **Advantage Calculation**: Compute the rewards-to-go and estimate the Advantage using the Critic.
- 3 **Optimization**: Perform epochs of Stochastic Gradient Descent (SGD) on collected data:
 - Calculate the ratio.
 - Apply the clipping formula.
 - Update the Actor by maximizing the clipped objective.
 - Update the Critic by minimizing the Mean Squared Error (MSE) between predicted values and actual returns.

Target Date Funds (TDF) vs Dynamic Policy (DP/RL)

Assume a TDF with access to the same three investments given before that uses the following glide path (a *static* strategy):

Age range	50–54	55–59	60–64	65–69	70–74	75–80
1. U.S. Stock	44%	40%	35%	28%	20%	18%
2. International Stock	29%	26%	23%	19%	13%	12%
3. U.S. Bond	27%	34%	42%	53%	67%	70%

Assume a 50 year old investor with \$100,000 at $t = 0$ annually invests $I_t = \$15,000 * 1.03^t$ until they retire at 65. Then they take out $I_t = -\$50,000 * 1.03^t$ until they are 80. The goal is to be solvent at the end.

The TDF solvency probability is **26.6%** (10,000 MC paths).
In contrast, RL solvency probability is **58.6%** (found in 1.2 secs).

Goals Optimization Engine (GOE^{TM})

“A class of efficient-frontier consistent algorithms that map/invert the risk-return view into mental accounting and goals-based views.”



Published in the *Journal of Investment Management*, Among Others



**HARRY M.
MARKOWITZ
AWARD**

Backed by 2018 Harry
Markowitz Award-Winning
Research



Patented Process



Winner of the MMI/Barron's
Industry Award for Disruption

Wealth Tech 100
Award



is a WealthTech100 company



WEALTHTECH100
2024

**The World's Most Innovative
WealthTech Companies**

www.WealthTech100.com

Ref: (1) [Dynamic Programming paper](#); (2) [MetaRL paper](#)

MetaRL Overview

- 1 A LLM-inspired **pre-trained** reinforcement learning (RL) approach that solves the multi-goal wealth management problem, versus solving from scratch with dynamic programming (DP).
- 2 A meta-model (MetaRL) is designed to compute portfolio selection and goal-taking decisions for a range of scenarios **purely in inference mode**, without requiring online training.
- 3 Meta-model training on **thousands of problem sets** is economical and completes within a few hours.
- 4 MetaRL handles large state spaces, which is not feasible with older methods like DP.
- 5 The trained model solves new GBWM problems in seconds per episode, the pure inference time is actually 13 milliseconds as compared to 7.9 seconds for DP on the same AWS C7 24xlarge machine, beating the solution time of DP. The average % efficiency compared to DP is 98%.

MetaRL: A snapshot summary

What

- Goal-based wealth management with multiple goals (defined by cost and utility)
- Pre-trained Meta-model (MetaRL) model computes portfolio and goal-taking decisions **purely in inference mode (zero-shot), without requiring online training**

Why

- Inference is **fast**: 10 ms per decision on average
- Approach is **scalable** to large state spaces
- It is **economical** to train, in a few hours
- Pure inference **accuracy is $\approx 97.8\%$ on average**, and could potentially be further improved with fast online fine-tuning

Hierarchical Actor-Critic Architecture

GoalAgent and PortfolioAgent are independent but sequential entities, where the output of the first influences the environment or state for the second.

Category	Actor Network	Critic Network
Input Dimension	26 ($5 + 3K$, $K = 7$)	26 ($5 + 3K$, $K = 7$)
Hidden Layers (Nodes)	256, 64, 16	64, 16
Activation (Intermediate)	tanh	tanh
Activation (Output)	sigmoid	linear
Output Type	Scalar (a_g or a_p)	Scalar (Value Function)
Hyperparameter	Value	
Learning Rate	10^{-4}	
PPO Clip Parameter (ϵ)	0.2	
Discount Rate (γ)	1.0	
Training Epochs	1000	
Episodes per Epoch	500	
Total Training Time	≈ 4 hours	

Sample training result

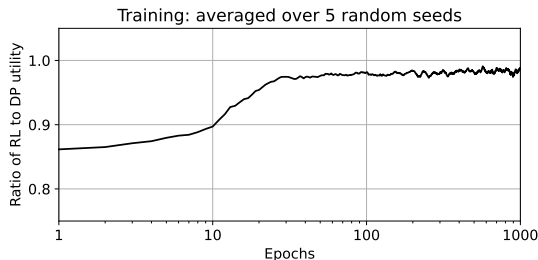


Figure: Performance during training, 1000 randomly generated problems averaged over 5 random seeds, with horizons up to 50 years. Training for each of the 1000 problems is done over 500 epochs. Shaded region indicates standard deviation.

The performance, denoted by the ratio of RL to DP utility, is fairly stable after 100 epochs

Case Studies: Samples

- ➊ Single-goal: Planning to purchase a house in 10 years.
 - ➋ Multi-goal: Planning to purchase a car in 3 years, Europe vacation in 5 years, and house in 15 years.
 - ➌ Retirement-goal: Planning the annual income for the retirement tenure, till life expectancy.
- Testing horizons go up to 100 years to evaluate out of distribution capability.
 - Number of goals varies from 1 to 60 with a mean of 38.

Table 9: Description of test cases with infusions. The base amount of infusion for odd-numbered test cases is $I = \frac{W(t)}{10(T-1)}$, rounded up to the nearest integer.

Case	T	W(t)	Non-zero goals $[t, C(t), U(t)]$	Non-zero infusions $[t, I(t)]$
34	10	100	[10, 150, 1]	[1, 10]
35	10	100	[10, 200, 1]	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
36	10	100	[10, 400, 1]	[1, 10]
37	40	100	[40, 600, 1]	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
38	40	100	[40, 1200, 1]	[6, 12]
39	40	100	[40, 2400, 1]	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
40	100	100	[100, 50000, 1]	[21, 19]
41	100	100	[100, 500000, 1]	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
42	100	100	[100, 200000, 1]	[27, 22]
43	3	100	[2, 75, 0.9], [3, 75, 1]	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
44	20	100	[10, 200, 1.3], [20, 500, 1]	[6, 12]
45	20	100	[15, 200, 1.3], [20, 300, 1]	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
46	35	100	[5, 50, 1], [25, 500, .5], [35, 1000, 1]	[13, 15]
47	60	100	[15, 300, .7], [30, 6000, 1.2], [45, 5000, .2], [60, 20000, 1]	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
48	25	100	[3, 30, .2], [5, 70, .3], [8, 70, .3], [25, 1000, 1]	[11, 14]
49	40	100	[10, 150, 1.5], [40, 600, 1], [35, 500, 1], [40, 600, 1]	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
50	20	100	$[t, 15, 1]$ for $t \in \{2, 4, 6, \dots, 20\}$	[10, 13]
51	20	100	$[t, 25, 1]$ for $t \in \{2, 4, 6, \dots, 20\}$	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
52	20	100	$[t, 50, 1]$ for $t \in \{2, 4, 6, \dots, 20\}$	[11, 14]
53	20	100	$[t, 75, 1]$ for $t \in \{2, 4, 6, \dots, 20\}$	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
54	60	100	$[t, 20, 1]$ for $t \in \{1, 2, 3, \dots, 60\}$	[58, 31]
55	60	100	$[t, t, 1]$ for $t \in \{1, 2, 3, \dots, 60\}$	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
56	60	100	$[t, t, 100 + t]$ for $t \in \{1, 2, 3, \dots, 60\}$	[41, 34]
57	60	100	$[t, t, 100 - t]$ for $t \in \{1, 2, 3, \dots, 60\}$	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
58	60	100	$[t, 60 - \frac{1}{2}]$ for $t \in \{1, 2, 3, \dots, 60\}$	[45, 38]
59	60	100	$[t, 60 - \frac{1}{2}, 100 + t]$ for $t \in \{1, 2, \dots, 60\}$	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
60	60	100	$[t, 60 - \frac{1}{2}, 100 - t]$ for $t \in \{1, 2, \dots, 60\}$	[49, 43]
61	30	100	[3, 35, 1.5], [6, 35, 1.3], [9, 5, 0.4], [12, 50, 1], [15, 15, 0.7], [18, 3, 0.3], [21, 45, 0.6], [24, 120, 0.9], [27, 170, 1.1], [30, 160, 1]	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
62	16	12	[16, 34.25, 26]	[14, 3]
63	16	21.63	[8, 18.50, 18], [16, 34.25, 26]	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
64	16	38.99	[4, 13.60, 14], [8, 15.87, 16], [12, 25.18, 22], [16, 34.25, 26]	[15, 6]
65	16	70.27	[2, 11.66, 12], [4, 13.60, 14], [6, 15.87, 16], [8, 18.50, 18], [10, 21.59, 20], [12, 25.18, 22], [14, 29.37, 24], [16, 34.25, 26]	$[t, 1.03^t I]$ for $t \in \{1, \dots, T-1\}$
66	16	126.67	[1, 10.8, 11], [2, 11.66, 12], [3, 12.60, 13], [4, 13.60, 14], [5, 14.69, 15], [6, 15.87, 16], [7, 17.14, 17], [8, 18.50, 18], [9, 19.99, 19], [10, 21.59, 20], [11, 23.32, 21], [12, 25.18, 22], [13, 27.20, 23], [14, 29.37, 24], [15, 31.72, 25], [16, 34.25, 26]	[16, 21]

Case 20: DP/RL vs. MetaRL

Scenario details:

① $T = 20$ years

② $W_0 = \$100$

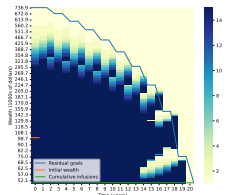
③ Goals:

① $t \in \{2, 4, 6, \dots\}$

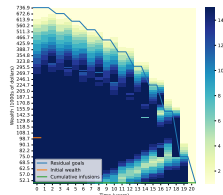
② $C(t) = \$75$

③ $U(t) = 1$

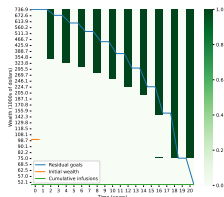
DP/RL: Portfolio policy



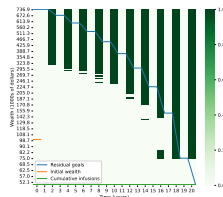
MetaRL: Portfolio policy



DP/RL: Goal-taking policy



MetaRL: Goal-taking policy

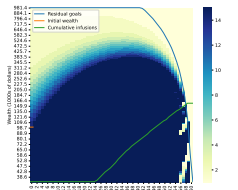


Case 57: DP/RL vs. MetaRL

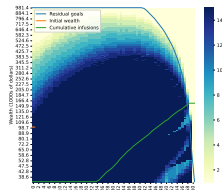
Scenario details:

- 1 $T = 20$ years
- 2 $W_0 = \$100$
- 3 Goals:
 - 1 $t \in \{1, 2, 3, \dots\}$
 - 2 $C(t) = \$t$
 - 3 $U(t) = 100 - t$
 - 4 total infusions of \$157 spread uniformly across the tenure.

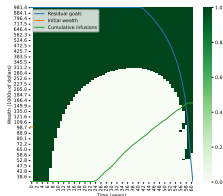
DP/RL: Portfolio policy



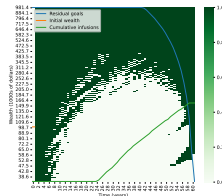
MetaRL: Portfolio policy



DP/RL: Goal-taking policy



MetaRL: Goal-taking policy



Results Summary

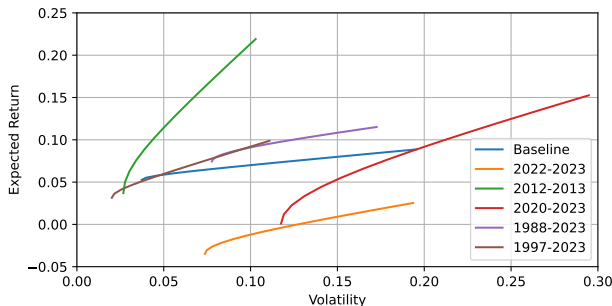
Time (in ms)	Mean	Std. dev.	Min.	Median	Max.
MetaRL inference	20.94	1.34	16.5	21.1	23.9
DP solution	2198	2255	83	1248	8012
Speed-up	$\approx 105x$		$\approx 5x$	$\approx 59x$	$\approx 335x$

Observations

- 1 In finding the optimal actions at current time, RL may provide upto 335x speed-up compared to RL/DP, depending on the scenario.
- 2 This is a significant saving in time as well as computational resource, given tens of thousands of investors per wealth management institution.

Robustness to Varying Efficient Frontiers

The MetaRL model was trained on the baseline efficient frontier!



Efficient Frontier	Mean RL-Efficiency
Baseline	0.978
2022-2023	0.986
2012-2013	0.995
2020-2023	0.990
1988-2023	0.984
1997-2023	0.987

- Proposal: Train a single reinforcement learning model to compute near-optimal goal-taking and portfolio selection decisions in a wide variety of investor scenarios, purely in inference mode
- Resulting meta RL framework is,
 - **fast:** 100x quicker than DP when computing decisions
 - **accurate:** 97.8% of optimal utility averaged over 66 test cases
 - **extensible:** ability to reuse existing model for unseen scenarios or increase dimension without affecting inference runtime

Top 10 RL use cases in finance

- 1 Optimal Trade Execution (Order Slicing)
- 2 Dynamic Portfolio Rebalancing
- 3 Market Making and Inventory Management
- 4 Continuous-Time Options Hedging
- 5 Algorithmic High-Frequency Trading (HFT)
- 6 Dynamic Risk Management & Stress Testing
- 7 Automated Credit Scoring & Loan Pricing
- 8 Crypto Yield Farming & DeFi Liquidity Routing
- 9 Robo-Advising & Goal-Based Wealth Planning
- 10 Fraud Detection & Anti-Money Laundering (AML)

Agents to Build RL Solutions

Three Pillars of Applied RL

- **Domain Expertise:** Understanding the specific problem domain and business context.
- **Problem Definition, MDP Design:** Structuring the core Markov Decision Process (MDP) by explicitly defining states, actions, rewards, and dynamics.
- **System Implementation:** Executing a runnable code repository to handle reward shaping, hyperparameter tuning, environment validation, model training, and final policy evaluation.

Chat Assisted Reinforcement Learning (CARL)

The Core Problem: The **Specification Gap**

CARL bridges the specification gap in complex applied RL.

- Implementation Stage: Modern agentic systems excel at coding and hyperparameter tuning. However, autonomous code generation addresses only the execution stage.
- Specification Stage: Failures often originate earlier, when human intent has not yet been translated into a precise problem definition or Markov Decision Process (MDP).

Design Principle: A correctly specified problem with a suboptimal algorithm can be improved incrementally; a correct implementation of the wrong MDP optimizes the wrong problem.

The RL Specification Gap & LLMs

- **The Root Failure:** Applied RL projects primarily fail due to underspecified problem definitions, not algorithmic shortcomings (Hamadani, 2022).
- **The Tool Gap:** Implementation suites (e.g., Stable-Baselines3, FinRL) inherently assume formal MDP/reward structures are already complete (Schulman et al., 2017; Liang et al., 2021; Liu et al., 2022).
- **LLM Limitations:** Standard assistants (e.g., ChatGPT) lack structured step-by-step design logic (Bai et al., 2024), leading to execution anomalies like *reward hacking* and *state aliasing*.

The "Vibe Coding" Failure

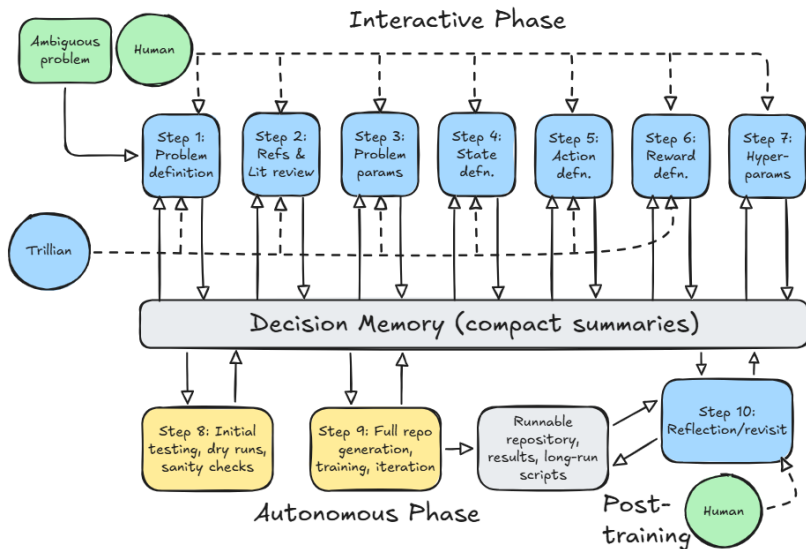
- Standard coding assistants often fail even simple financial problems like the Merton consumption-investment task.
- While the code runs "bug-free," results often show glaring inaccuracies when checked against closed-form solutions.
- Without structured MDP design, agents find "reward hacks" or suffer from "state aliasing."

Introducing CARL

- Targeted Solution: Chat Assisted Reinforcement Learning (CARL) is an architecture specifically engineered to bridge the specification gap.
- Human-Agent Co-Design: CARL pairs a human domain expert with an AI assistant to iteratively map out complex business goals into rigorous mathematical concepts before any programming begins.
- Empirical Validation: When tested on realistic challenges like dynamic portfolio optimization, CARL prevents "vibe coding" errors and produces policies that match or exceed analytical benchmarks.

- Interactive Phase (Steps 1–7): A structured, seven-step conversational workflow that forces sequential decisions on every element of the MDP.
 - Bespoke Skill Files: Rather than loading the entire system prompt at once, CARL leverages isolated markdown "skill files" to guide the user through one discrete design decision at a time.
 - Context Management: Each completed step is compressed into a compact summary, carrying key design choices forward without overwhelming the LLM's context window.
 - Generates, debugs, and iterates on training pipelines
- Autonomous Phase (Steps 8–9): Once the design is locked in, CARL enters an automated tool-use loop to generate, execute, test, and debug a fully functional RL pipeline without requiring ongoing human input.

Logical Design of CARL



Problem Definition

**CARL** Chat Assisted Reinforcement Learning

Step 1/9: Problem Definition

Current step

/home/sagemaker-user/

Thinking pane

Restart

CARL's questions

Step 1: Problem Definition

What problem are you looking to solve with RL? A brief description of the domain and what you're trying to achieve is a good starting point.

User response

I want to solve the problem of allocating wealth between a stock and a bond, and maximizing the utility of intermittent financial goals. Use refs/Stocks_Bonds.md to read about it.

I've read the reference. Before summarizing, I want to reconcile something between your opening description and the document.

- Your message mentions "utility of intermittent financial goals" – i.e., possibly multiple goals occurring at different times during the horizon, with a utility function over goal attainment.
- The reference document describes a single terminal goal G at time T with the objective $\mathbb{P}(W_T \geq G)$ – no intermediate goals, no utility function.

These are meaningfully different formulations. A few clarifying questions:

- Which framing do you want to pursue?
 - The document's version: single terminal goal, max $\mathbb{P}(W_T \geq G)$.
 - Multiple intermediate goals at various times $t_1, t_2, \dots, t_N$, each with a utility (e.g., sum of utilities from goals met, with wealth withdrawn when a goal is funded).
 - Something else / hybrid.
- For the second asset, which variant do you want to star

Trillion pre-filled suggestion in editable box

1. Single terminal goal at time T , maximize $\mathbb{P}(W_T \geq G)$.

2. Start with VTI and BND.]

Send

THINKING

-- Step 1: Problem Definition --

What problem are you looking to solve with RL? A brief description of the domain and what you're trying to achieve is a good starting point.[tokens: in=1670 (cached=0, read=0) out=43]

I've read the reference. Before summarizing, I want to reconcile something between your opening description and the document.

- Your message mentions "utility of intermittent financial goals" – i.e., possibly multiple goals occurring at different times during the horizon, with a utility function over goal attainment.

- The reference document describes a "single terminal goal G at time T , with the objective $\mathbb{P}(W_T \geq G)$ – no intermediate goals, no utility function.

These are meaningfully different formulations. A few clarifying questions:

1. Which framing do you want to pursue?

(a) The document's version: single terminal goal, max $\mathbb{P}(W_T \geq G)$.

(b) Multiple intermediate goals at various times $t_1, t_2, \dots, t_N$, each with a utility (e.g., sum of utilities from goals met, with wealth withdrawn when a goal is funded).

- (c) Something else / hybrid.

2. For the second asset, which variant do you want to star

Step 6 (Reward Definition) Skill

Condensed excerpt from the Step 6 skill, with a local objective, design hints, and checklist items.

Skill element	Excerpt from Step 6: Reward Definition
Description	Design the reward with normalization and domain randomization; ask for hints or nudges for early training.
Design hints	Keep the reward a single readable expression; drop fixed constants; expose reward components in <code>info</code> ; document discounting.
Reward checks	Complete reward-configuration and credit-coverage tables; ensure terminal/failure terms reach responsible agents; label unmatched oracle formulas as heuristic.
Randomization checks	Randomize policy-relevant parameters at reset; include randomized parameters in the observation; disable randomization during evaluation.

Step 6 Decision Summary

Step 6: Reward Definition, from `memory.md` of one of the CARL runs in the results section

- **Reward function:** Sparse terminal only, $R = \sum_{i \in \text{taken}} u_i (1 + 0.25 k_i)$ at $t = T$; zero at all other timesteps.
 - **Reward range:** $[0, \approx 15.6]$; no normalization, clipping, log, or tanh compression.
 - **Discount:** RL $\gamma = 1$ (finite horizon, terminal-only reward).
 - **Goal PV discounting:** $r = 0.08$ from Step 4 lives in state features, not in the reward.
 - **Credit assignment:** Both Agent A (DQN) and Agent B (PPO) receive the same scalar terminal R .
 - **Shaping:** None.
 - **Warm-starts / pretraining:** None.
 - **Info dict per step:** `wealth`, `log_wealth`, `k`, goal masks, due/affordable flags, `action_A`, `action_B`.
 - **Terminal info:** `terminal_reward`, `utility_taken_raw`, `bonus_component`, `goals_taken_count`.
 - **Pathological-policy checks:** skip-all, greedy-take, and all-stocks should score below DP baseline.
 - **Domain randomization:** Step 3 parameters appear in observations, are disabled in canonical eval, and are logged.
 - **User directive:** no shaping.
 - **User directive:** no DP warmups; DP is evaluation-only with no policy-training leakage.
 - **User directive:** use $\gamma = 1$.
-

Step 7



CARL Chat Assisted Reinforcement Learning

Step 8/9: Initial Testing

/home/sagemaker-user/CARL/sessions/20260703-1556

Restart

STEP 7 SUMMARY

- **Algorithms:** PPO, $\gamma = 1$, GAE $\lambda = 0.95$, clip $\epsilon = 0.2$, 4 epochs per update
- **Batch:** 500 episodes/update (~7,500 transitions at avg $T = 15$), mini-batch 512, learning rate $3e-4$
- **Value loss:** Huber (smooth_l1); critic head sigmoid-bounded to $[0, 1]$
- **Advantage normalization:** none (--adv-norm none) - block-centering replaces it
- **Entropy coefficient:** linear anneal $0.01 \rightarrow 0.001$ per update round
- **Action head:** $\text{Beta}(\alpha, \beta)$ via $1 + \text{softplus}$; final-layer biases initialized so $\alpha \approx \beta \approx 2$ (mean $w_{\text{VPI}} = 0.5$, std ≈ 0.22)
- **Init diagnostic:** print sampled w_{VPI} quantiles (5/50/95%) on representative states ($\tau = 20, W/G = 1$), ($\tau = 5, W/G = 0.5$), ($\tau = 5, W/G = 1.5$) before training
- **Parameter stability:** EPISODES_PER_PARAMETER_SET = 10; 50 parameter sets per update ($50 \times 10 = 500$)
- **CRN within blocks:** all 10 episodes in a block share the same VTI+BND shock path; block-centered advantages applied to advantage only (value targets remain raw returns)
- **Antithetic variates:** enabled within CRN blocks, 5 $(Z, -Z)$ pairs per 10-episode block
- **Multi-parameter CRN sweep:** enabled on $\$*(\text{primarydifficultydriver})\text{sweep}^*$ \in {0.01, 0.03, 0.05, 0.07, 0.09} with 2 antithetic reps at fixed shock path and γ
- **Evaluation:** independent noise paths, no CRN, no antithetic
- **Network architecture:** actor and critic each [256, 256] MLPs with tanh activations, separate networks
- **No heuristic-residual actor:** constant-mix baseline too weak to anchor to; end-to-end RL
- **Config & reproducibility:** all hyperparameters centralized in config.py; seeds set for numpy, torch, env RNGs; separate eval seed offset from train seed
- **Time limits:**
 - Step 8 (Initial Testing): 15 minutes
 - Step 9 (Launch / Full Training): 30 minutes

Starting autonomous execution. You can send a message at any time to guide CARL.

Step 8: Initial Testing

Send a message to guide CARL...

Interject

THINKING

... Step 7 might be fairly representative, but I need to verify in Step 6" but they haven't been checked yet).

Plan for Step 8 (tight 15-min budget)

1. Build minimal repo: 'config.py', 'env.py', 'features.py', 'heuristic.py', 'networks.py', 'dry_run.py', 'pretrain.py', 'tests/'.
2. Dry run: 500 random episodes + reward/state ranges, verify clip bounds.
3. Heuristic sweep (constant-mix on grid) + establish >rand dom baseline.
4. Reward-decomposition trace on 3 deterministic scenarios.
5. Supervised pre-train actor to imitate a difficulty-aware heuristic; skip critic warmup (terminal-only binary reward makes critic warmup on the pretrained policy a natural fit but I'll include a short warmup).
6. Env + agent pytest.

```
Let me build.[calling write_file...]
[calling write_file...]
[calling write_file...]
[calling write_file...]
[tokens: in=6526 (cached=0, read=9702) out=5068]
[write_file: config.py]
-> File written: code/config.py (1384 chars)
[write_file: features.py]
-> File written: code/features.py (654 chars)
[write_file: env.py]
-> File written: code/env.py (4884 chars)
[write_file: heuristic.py]
-> File written: code/heuristic.py (1542 chars)
[calling write_file...]
```

Auxilliary Features

- **Brainstorming mode:** Activates via `/suggest` in Step 1 to search literature and synthesize candidate formulations for users with only broad domain interests. The chosen formulation becomes a reference document routed into the standard workflow.
- **Step 10 revisit mode:** Reopens completed sessions for post-training diagnosis by reviewing memory, logs, plots, and code. Proposed changes are discussed interactively and prepared in a fresh workspace rather than altering the completed run.
- **Autonomous-step interjections:** Allows users to inject guidance during the autonomous generation, testing, or training phases. Messages are added to the stream to redirect implementation without breaking step boundaries or restarting the workflow.
- **Trillian persona update:** Evaluates accepted or edited responses to update the simulator's domain-expert profile after a session. Uses a conservative three-stage pipeline (triage, minimal edits, consistency checks) to prevent persona drift.
- **L^AT_EX report output:** Compiles a paper-style document from accumulated design memory, execution artifacts, logs, plots, and literature once a session completes. Serves as official workflow documentation rather than an active part of the MDP design.

Interactive Decision-Making

**CARL** Chat Assisted Reinforcement Learning Step 4/9: Information Proposal

/home/sagemaker-user/CARL/sessions/20260703-1556 [Restart](#)

Explain the rationale behind the wealth to goal ratio. Should we not discount the goal value to the present value?



Rationale for W_t/G (and the discounting question)

Why W_t/G at all

The core reason is **scale invariance of the objective**. The success event is:

$$W_T \geq G \iff \frac{W_T}{G} \geq 1$$

Because both VTI (GBM) and the portfolio's evolution are multiplicative in wealth, the dynamics of W_t/G under any policy depend only on:

- the ratio W_t/G itself,
- time remaining τ ,
- the bond state B_t ,
- the action.

They do **not** depend on the absolute levels of W_t or G separately. So (W_t, G) as two features carries a redundant degree of freedom – the ratio is a sufficient statistic. This is the “ratio not raw values” principle from the skill.

It also keeps the feature bounded and scale-free across the full $G \in [110, 800]$ range from Step 3, so the network sees the same input distribution regardless of episode difficulty.

[Send](#)

THINKING

The role that discounting would play – telling the policy “you have $\log(G/W_t)$ years to close the gap” – is done explicitly by $\log(G/W_t)$ (the required log-return per year). This is cleaner than folding time into G , because it separates:

- **level** ($\log(G/W_t)$: how far am I from the goal today?)
- **rate** ($\log(G/W_t)$: what annualized return do I need?)

These decouple two structurally distinct signals, which the skill explicitly recommends.

4. If we did discount, at what rate?

The only defensible rates would be **BND** return – but these are “policy-dependent” and “asset-choice-dependent”. Discounting at $\log(G/W_t)$ would essentially pre-bake an all-stock strategy into the state. That’s leaking an analytical answer into the input, which the skill warns against.

Summary

Keep $\log(G/W_t)$ (or its log) undiscounted. The time dimension enters cleanly through $\log(G/W_t)$ and the required-return feature. Discounting would either require an arbitrary rate choice or bias the state toward a particular policy.

Does this resolve the question? And returning to the open questions: shall we go with $\log(G/W_t)$ only (dropping the raw G), and $(B_t - \theta)/\theta$ for the bond level? [tokens: in=5025 (cached=0, read=0) out=1247]

CARL previously proposed a state space definition in Step 4. User queries CARL regarding the rationale behind one of the steps, to which CARL provides an answer. The user may choose to accept, modify, or reject the proposal until a final agreement is reached.

Thinking Pane

```
THINKING
-
-> File written: code/long_train.sh (2003 chars)
[calling run_command...]
[tokens: in=43474 (cached=0, read=14782) out=84]
[run_command: chmod +x long_train.sh && ls -la long_train.sh]
-> -rw-r--r-- 1 sagemaker-user users 2005 Jul 3 16:23 long_train.sh
Now: (1) save the pretrained model as 'experiments/pretrain/init_actor.pt' for parity with 'long_train.sh', (2) quick tests, (3) launch training.[calling run_command...]
[tokens: in=43612 (cached=0, read=14782) out=177]
[run_command: mkdir -p experiments/pretrain && cp pretrain.ed.pt experiments/pretrain/init_actor]
-> total 536 drwxr-xr-x. 2 sagemaker-user users 27 Jul 3 16:23
drwxr-xr-x. 3 sagemaker-user users 22 Jul 3 16:23
.. -rw-r--r-- 1 sagemaker-user users 546613 Jul 3 16:23 init_actor.pt
[calling run_command...]
[tokens: in=43919 (cached=0, read=14782) out=199]
[run_command: python -c "
import torch
from agent import PPOAgent
a = PPOAgent()
a.load('exper')
-> load OK action=0.0236 logp=2.1613 v=0.8011
[calling run_command...]
[tokens: in=44155 (cached=0, read=14782) out=85]
[run_command: python -m pytest tests/ -q 2>&1 | tail -20]
-> .....
[100%] 15 passed in 4.00s
Tests pass. Quick smoke check of 'train.py' with a tiny run (3 updates), then launch the full 30-minute run in the background.[calling run_command...]
[tokens: in=44276 (cached=0, read=14782) out=197]
[run_command: python train.py --run-name smoke --updates 2
```

```
THINKING
-
- Critic warmup: 30 updates x ~3s = ~90s
- PPO: ~100 updates x ~15s = 1500s
- Eval overhead: ~5s per eval x 20 = 100s
Total ~1700s = 28min. Let's launch.[calling run_command...]
[tokens: in=45096 (cached=0, read=14782) out=236]
[run_command: rm -rf experiments/smoke]
->
[calling run_background...]
[tokens: in=45344 (cached=0, read=14782) out=171]
[run_background: python -u train.py --run-name gbm_main -updates 100 --critic-warmup 30 --eval-]
-> Started (PID 7493). Output streaming to code/train_main.log. Use check_process to monitor.
[calling run_command...]
[tokens: in=45560 (cached=0, read=14782) out=70]
[run_command: sleep 90 && date]
-> Fri Jul 3 16:26:47 UTC 2026
[calling check_process...]
[tokens: in=45663 (cached=0, read=14782) out=94]
[check_process: train_main.log]
[waiting 90s before checking...]
-> Status: running (PID 7493) Log (train_main.log) last 40 lines: [init] Loaded experiments/pretrain/init_actor.pt [eval] Fixed eval set: 200 episodes. == Reward-buffer regression tests == 20/20 episodes pass reward-buffer checks.
[eval:init] {'success_rate': 0.765, 'W_T_mean': 297.243193 9238901, 'a
Training is progressing. Init success 0.765 (deterministic on 200 eps), ep_ret already climbing 0.738 + 0.822 by update 4. Let me let it run and check periodically.[calling run_command...]
[tokens: in=46688 (cached=0, read=14782) out=135]
[run_command: sleep 300 && date]
```

LHS: CARL's thinking trace as it sets up the directory structure and completes dry runs in preparation for full training. RHS: signing the training run based on available compute time, and is monitoring training as it progresses through the initial steps.

System Architecture & Technology Stack

• Lightweight Python Orchestration Layer

- Replaces monolithic platforms with modular components (routes, conversation engine, session layer).
- Supports a Flask web app and terminal client sharing a directory-backed session model.
- Asynchronous background threads handle intensive model loops and training tasks.

• Multi-Model Role Assignment

Model Role	Assigned Engine	Thinking Setting
Main Workflow	Claude Opus 4.7	Variable (Med → Highest)
Trillian Suggestion	Claude Haiku 4.5	Narrow / Low-cost
Report & Editorial	Claude Sonnet 4.6	Parallel structured prose

• Model-Agnostic Conversational Orchestration

- Interfaced via Bedrock Converse API, but logic remains completely uncoupled from any single vendor.
- CARL controls step boundaries, context policies, tools, and state summaries.

Market Equations & Transitions

Equity Dynamics (GBM)

$$dS_t = \mu_S S_t dt + \sigma_S S_t dM_t^{(S)} \implies S_{t+1} = S_t \exp \left(\left(\mu_S - \frac{1}{2} \sigma_S^2 \right) + \sigma_S Z_{t+1}^{(S)} \right)$$

Where $\mu_S = 0.128$, $\sigma_S = 0.201$, and $S_0 = 373$.

Bond Dynamics (Ornstein-Uhlenbeck Transition)

$$B_{t+1} = B_t e^{-\kappa_B n_h} + \theta_B (1 - e^{-\kappa_B n_h}) \\ + \sigma_B \sqrt{\frac{1 - e^{-2\kappa_B n_h}}{2\kappa_B}} Z_{t+1}^{(B)}$$

Where $\kappa_B = 5.20 \times 10^{-4}$, $\theta_B = 73.3$, $\sigma_B = 0.200$, and $B_0 = 73.2$.

- **Correlated Discrete Shocks:** $Z_{t+1}^{(B)} = \rho Z_{t+1}^{(S)} + \sqrt{1 - \rho^2} \xi_{t+1}$ ($\rho = 0.0204$)

The Secret Sauce: Conditional Utility Bonuses

Wealth Evolution Engine

$$W_{t+1} = (W_t - a_{i,t} G_i) R_{t+1}(\pi_t) \quad \text{where } R_{t+1}(\pi_t) = \pi_t \frac{S_{t+1}}{S_t} + (1 - \pi_t) \frac{B_{t+1}}{B_t}$$

Conditional Path-Dependent Rewards:

- If goal i has a base utility U_i , and k_i earlier goals have already been taken, the realized utility scales linearly:

$$\tilde{U}_i = U_i(1 + 0.25k_i)$$

- **The Core RL Optimization Trade-off:** Maximize $\mathbb{E} \left[\sum_i \tilde{U}_i \mathbf{1}\{\text{goal } i \text{ funded}\} \right]$.
- **Why DP Fails Here:** Early low-utility goals act as multipliers for massive terminal goals. This deep path-dependency breaks traditional standard backward-induction dynamic programming solutions.

Evaluation Matrix: Canonical vs. Reversed

Scenario	Goal Years	Amounts (G_i)	Base Utilities (U_i)
canon_1goal	{16}	{300}	{8}
canon_2goal	{8, 16}	{150, 150}	{3, 5}
canon_3goal	{5, 10, 16}	{125, 125, 125}	{1, 2, 5}
canon_4goal	{4, 8, 12, 16}	{100, 100, 100, 100}	{1, 1, 2, 4}
rev_2goal	{8, 16}	{150, 150}	{5, 3}
rev_3goal	{5, 10, 16}	{125, 125, 125}	{5, 2, 1}
rev_4goal	{4, 8, 12, 16}	{100, 100, 100, 100}	{4, 2, 1, 1}

Table: The Canonical cases stack utility heavily toward the terminal horizon (bonus seeking); Reversed cases demand heavy, immediate early funding decisions.

The Evaluation Methodology

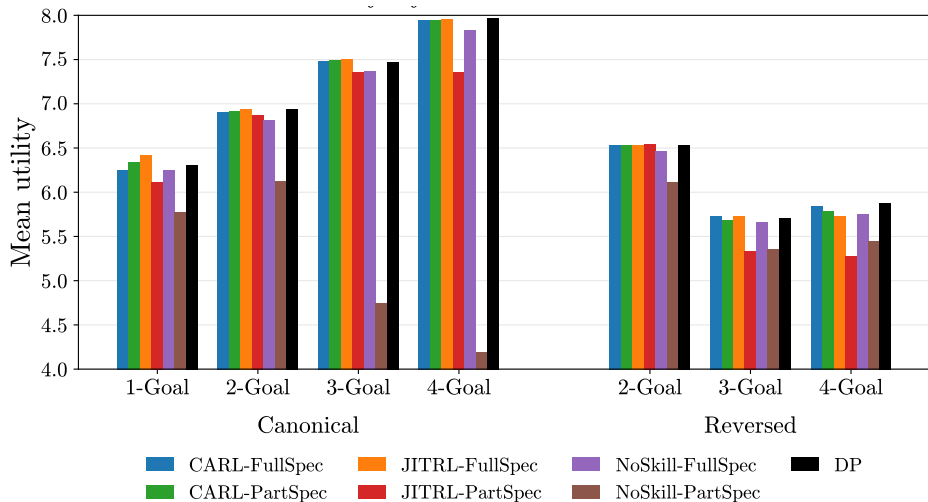
1. The Specification Axis

- **FullSpec:** Agent gets exact details (bounds, utility constraints, terminal rules, exact step-mechanics).
- **PartSpec:** Agent gets only a high-level domain description brief. Must infer/decide deep MDP details alone.

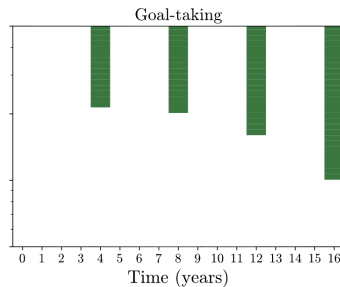
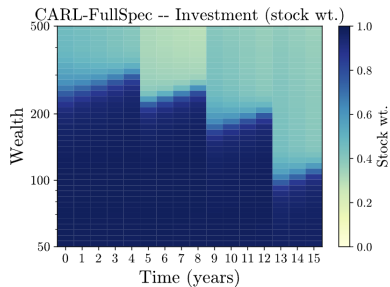
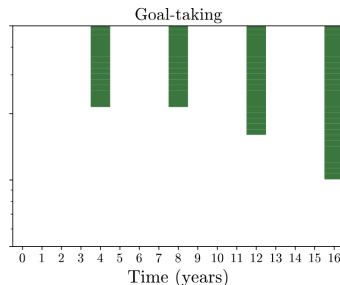
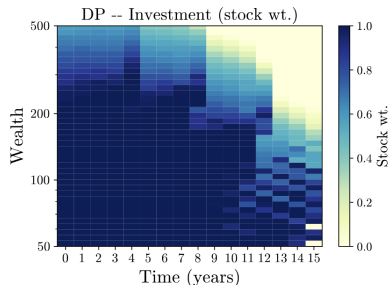
2. The Skill-Access Axis

- **CARL:** Uses a structured human clarification dialogue loop to lock down details.
- **JITRL:** Agentic loop with code skills but *no conversation capabilities* to resolve ambiguities.
- **NoSkill:** Plain-prompt coding with zero pipeline design frameworks.
- **DP Reference:** Backward induction with a simplified GBM model.

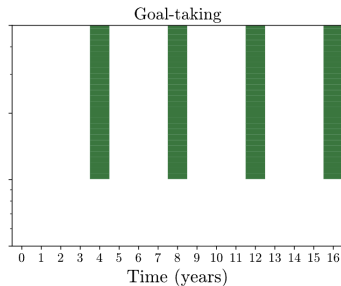
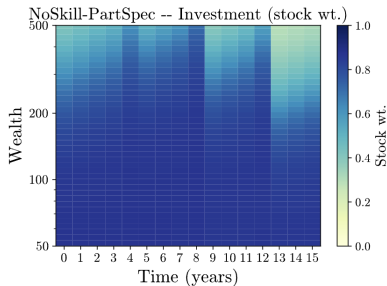
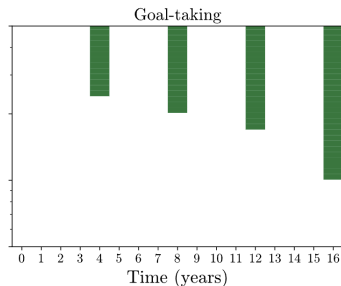
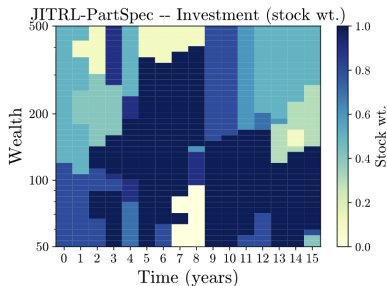
Utility by Method and Scenario



Action Spaces



Action Spaces



Core Architectural Conclusions

- ① **The Value of Staged Dialogue:** Conversational scaffolding converts incomplete briefs into optimal mathematical structures without requiring complete specifications upfront.
- ② **Specification Quality vs. Coding Speed:** Poor agent performance is rarely caused by hyperparameters or basic coding errors; it stems from structural oversights in the MDP design itself.
- ③ **Iterative Pipeline Reality:** Rather than expecting perfect specification upfront, complex engineering requires tools that can capture adjustments during the realization phase.

AI Agents in Finance

- Always use a Sandbox
- Heterogenous Context *is* the product
 - ① SEC filings, earnings calls, press releases, market/alternative data, internal memos, news, broker research, etc.
 - ② Normalize financial data to Markdown, CSV, JSON
 - ③ Metadata is key
 - ④ Chunking strategy
 - ⑤ Multimodality
- SEC filings are hard to deal with or “adversarial”: parsing these is hard.
- Skills are the secret sauce
 - ① Any one can build them
 - ② Skills are better than code
 - ③ Skills need to be redesigned as models improve
 - ④ Skills give feedback to AI Labs and eventually become obsolete

- s3 is the new AI architecture: YAMLS, durable, simple, cheap.
- File system tools are essential. SQL tools are a must across financial databases.
- Checkpointing is critical for long-running agentic workflows. Retries are an essential aspect of non-deterministic programming.
- Streaming data. Reactive agents are needed.
- Interactive agents are key in finance (see CARL for example)
- Agent evaluation is non-optional. Can agents disambiguate tickers, attribute fiscal periods correctly, understand numerical precision and financial units, avoid hallucinations?
- “The model is not your product. The experience around the model is your product.”

Token Economics: Companies must turn tokens into outcomes!

The stack:

- $\text{AI Outcome} = \text{Tokens} \times \text{Intelligent Operating Model} (= \text{observability, autonomy, deployment speed, accountability})$
- $\text{AI Spend} = \text{users} \times \text{tasks/user} \times \text{tokens/task} \times \$/\text{token}$
- $\text{AI Value} = \text{tasks completed} \times \text{value per task}$

Observability – we need to get better at quantification of each component of the value chain.

“AI is overhyped in the short term and drastically underestimated in the long term.” – Demis Hassabis

We discussed the following four topics:

- Agentic AI – Escape Capability overhang
- RL for Retirement planning
- AI Agents to solve the RL Specification Gap and democratize RL solutions
- Tips on Agentic AI in Finance

THANK YOU!